Informe: Trabajo Práctico N2

Pruebas Automatizadas y Diseño UI

FerreRAP – Sistema de Gestión de Stock para Ferretería

Ingeniería de Software 2

Universidad de la Cuenca del Plata

Posadas, Misiones

2026

Integrantes:

Integrante	**Rol**	**GitHub**
Santiago Gonzalez	Scrum Master	@santi-ngonzalez
Luciano Fohrholtz	Dev Lead	@LuchoFohrholtz
Juan Carlos Abente	QA Lead	@juankiabente
Mariano Acosta	UX Lead	@mittax6

Índice

Introducción y contexto **3**
1.1 Equipo y roles 3
1.2 Repositorio 4
Parte A - Diseño de Interfaz Centrado en el Usuario **5**
- A1 – Prototipo en Figma 5
- A2 – Análisis de usuario, tarea y contexto 7
- A3 – Auditoría de usabilidad ISO 9241-11 8
Parte B – Pruebas de Software **12**
- B0 – Investigación previa: técnicas de diseño 12
- B1 – Casos de prueba unitaria 13
- B2 – Framework de pruebas y automatización CI/CD 16
- B3 — Diseño conceptual de pruebas de integración 19
Documentación, uso de IA y estado del repositorio **21**
Tablero Kanban 21
Uso de inteligencia artificial 22
Conclusión **23**

Introducción y contexto

El presente informe corresponde al Trabajo Práctico N°2 de la materia Ingeniería de Software II, dictada en la Universidad de la Cuenca del Plata durante el ciclo lectivo 2026. El trabajo fue desarrollado por el equipo Nipintucu, conformado por cuatro integrantes con roles diferenciados dentro de un marco metodológico Scrum.

FerreRAP es un sistema web de gestión de stock para una ferretería mediana, que evolucionó desde el TP1 incorporando una base de datos relacional en la nube (Supabase), un sistema de login con dos roles (Administrador y Empleado), un módulo de carrito de ventas con facturación, y exportación de reportes a PDF y Excel. La arquitectura se mantiene basada en los patrones de diseño Observer y Strategy implementados en el TP1.

El TP2 integra dos ejes temáticos del programa. El Eje 3 corresponde al Diseño de Interfaz centrado en el usuario, donde se formaliza el prototipo de FerreRAP en Figma y se realiza una auditoría de usabilidad según la norma ISO 9241-11. El Eje 4 corresponde a las Pruebas de Software, donde se aplican técnicas de partición de equivalencia y valores límite sobre el código existente, se implementan seis casos de prueba unitaria con pytest, y se configura un pipeline de integración continua mediante GitHub Actions.

1.1 Equipo y roles

Integrante	**Rol**	**Responsabilidades en el TP2**
Santiago Gonzalez	Scrum Master	Coordinación del equipo, redacción de tp2-ui-ux.md, actualización del AI_LOG, compilación del informe final.
Luciano Fohrholtz	Dev Lead	Implementación de los tests unitarios, configuración del pipeline GitHub Actions, evidencia audiovisual.
Juan Carlos	QA Lead	Investigación previa, redacción de

Abente		tp2-pruebas-unitarias.md, diseño conceptual de pruebas de integración.
Mariano Acosta	UX Lead	Diseño y publicación del prototipo en Figma con las tres pantallas del caso de uso principal.

1.2 Repositorio

El código completo, la documentación y todas las evidencias del TP2 están disponibles en el repositorio público del equipo en GitHub: *github.com/LuchoFohrholtz/is2-2026-nipintucu*

Parte A - Diseño de Interfaz Centrado en el Usuario

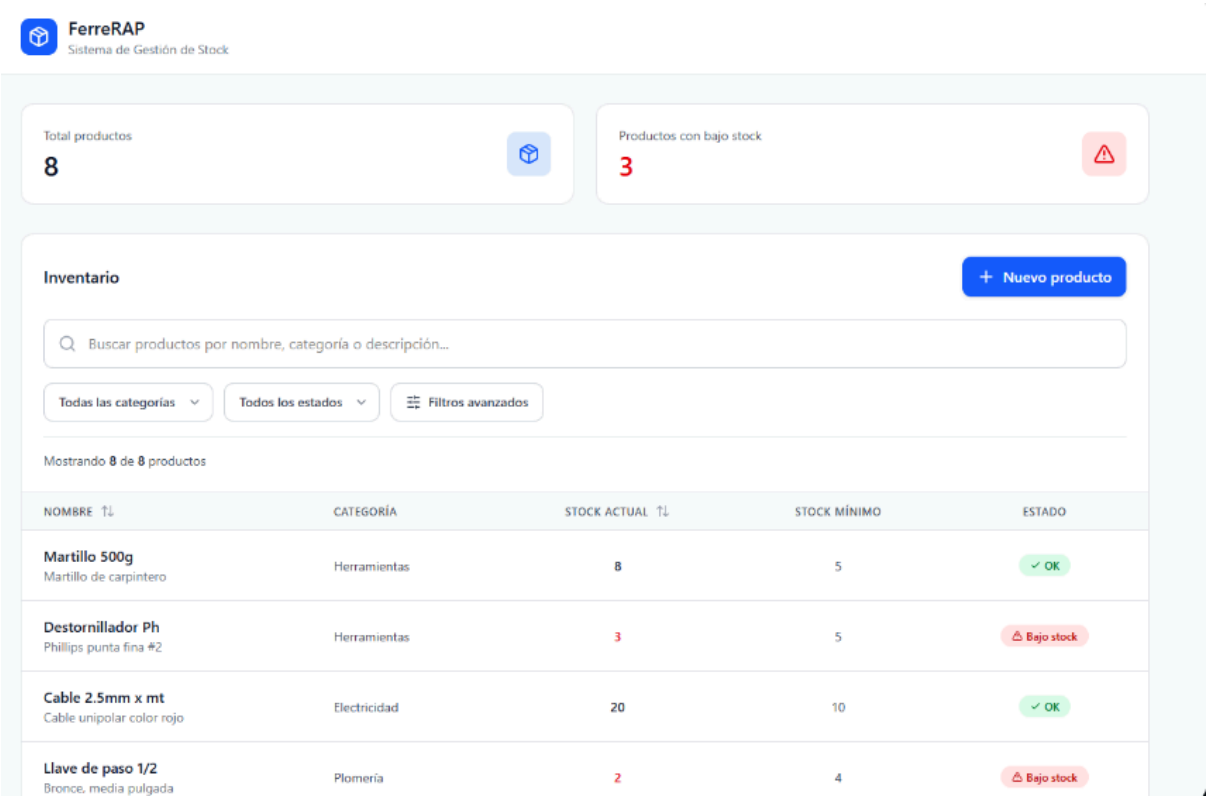
- *A1 – Prototipo en Figma*

El prototipo de FerreRAP fue desarrollado en Figma por el UX Lead. Es navegable y cubre el flujo completo del caso de uso principal del sistema: agregar un producto nuevo al inventario.

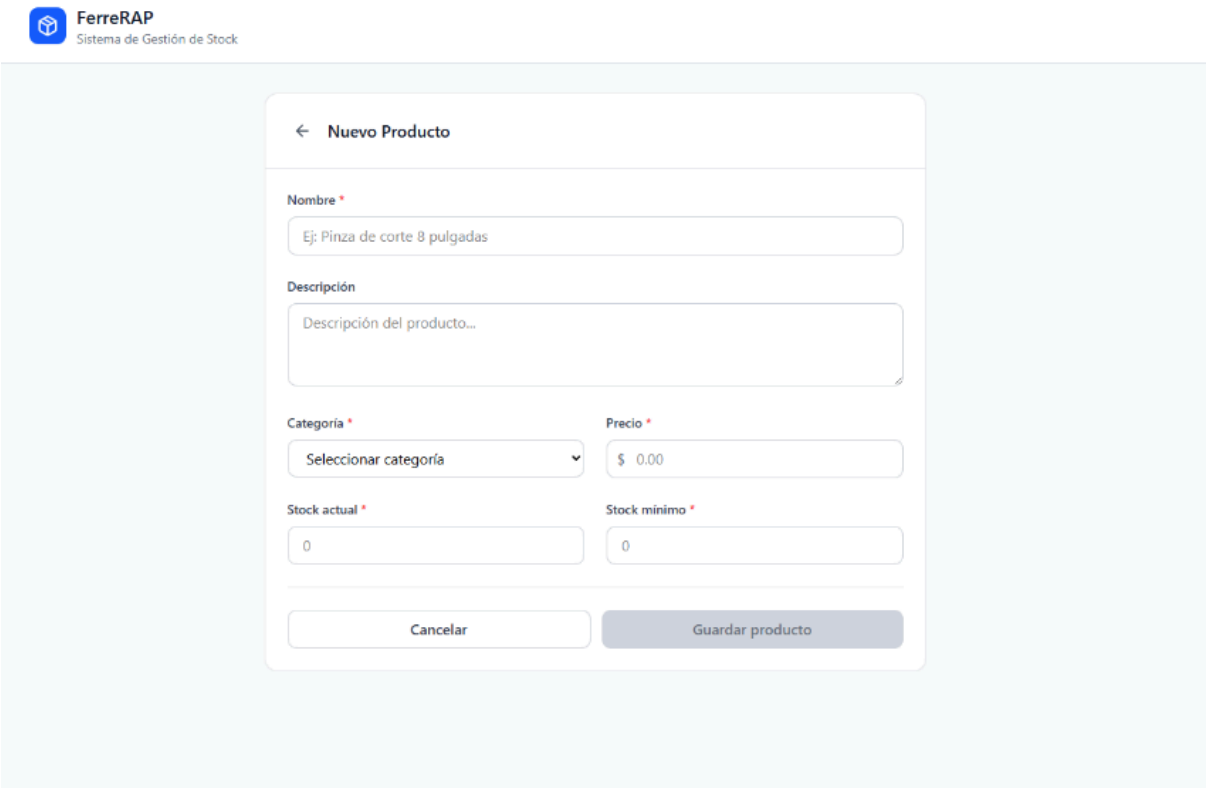
Link Prototipo: *https://www.figma.com/make/CicSoOUwMEUprpSG9INYkZ/FerreRAP-Stock-Management-Prototype?t=ufxEOo6hIVvvNYoi-1*

Pantallas incluidas:

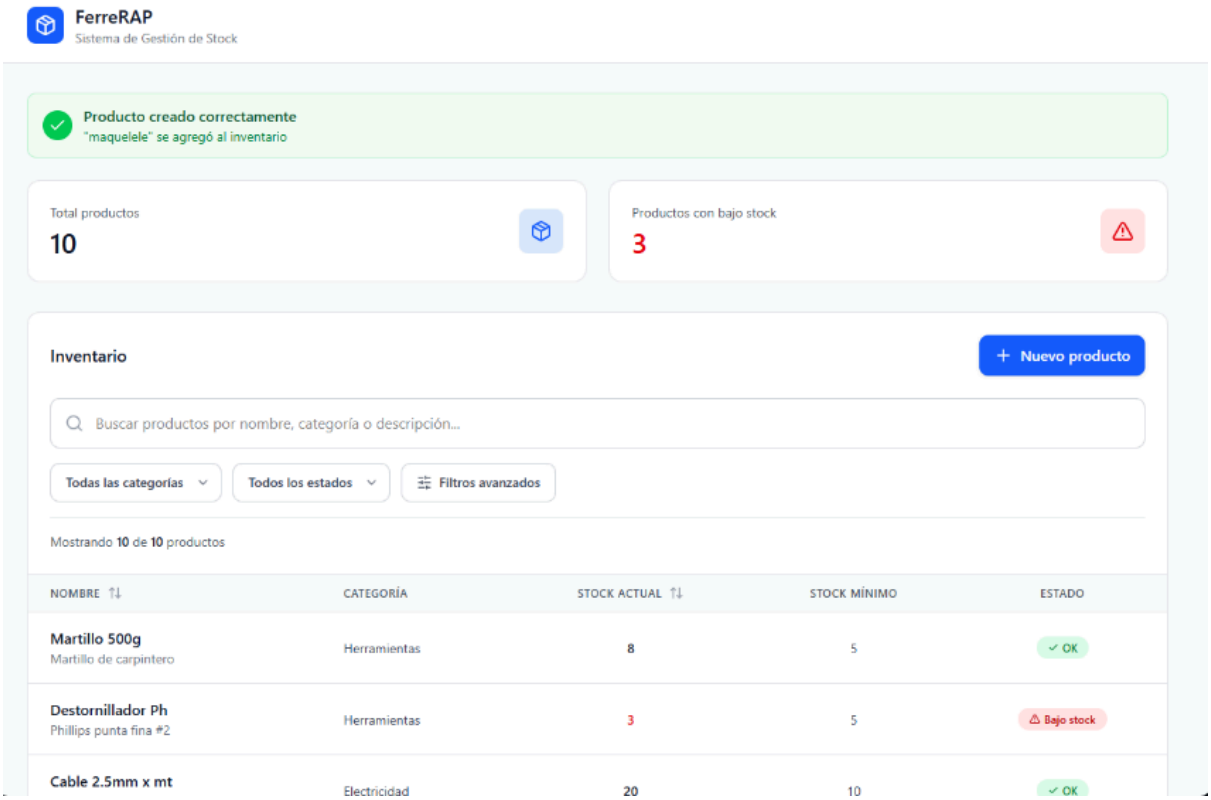
- Pantalla 1: Listado de productos con buscador, filtros por categoría y estado de stock visible mediante badges de colores.



- Pantalla 2: Alta de producto nuevo con formulario completo (nombre, descripción, categoría, precio, stock actual y stock mínimo).



- Pantalla 3: Vista del inventario actualizado con notificación de confirmación verde y contador de productos actualizado en tiempo real



Las capturas de las tres pantallas del prototipo estan publicadas en el repositorio del proyecto, dentro del archivo docs/tp2-ui-ux.md.: *https://github.com/LuchoFohrholtz/is2-2026-nipintucu/blob/main/docs/tp2-ui-ux.md#a1--prototipo-en-figma*

- *A2 – Análisis de usuario, tarea y contexto*

Usuarios objetivo

FerreRAP está pensado para dos tipos de usuarios bien distintos dentro de una ferretería mediana. Por un lado está el empleado de mostrador, que es quien más interactúa con el sistema en el día a día. Es una persona que no necesariamente tiene experiencia con software de gestión, y que trabaja bajo presión cuando hay clientes esperando. Por eso necesita que el sistema sea directo: entrar, registrar la venta, seguir atendiendo. No tiene tiempo para leer instrucciones ni recordar rutas complicadas dentro de una interfaz.

El segundo usuario es el encargado de compras o administrador, que usa el sistema de forma menos frecuente pero con mayor profundidad. Su foco está en revisar el estado del stock, analizar qué productos están por agotarse, generar reportes y decidir qué reponer. A diferencia del empleado, este usuario sí puede tomarse más tiempo para explorar el sistema, pero igual espera que la información esté organizada y sea fácil de interpretar de un vistazo.

Tareas principales

Las tareas que más se repiten en el sistema son: registrar una venta (salida de stock), consultar el inventario actual, revisar las alertas de productos con stock bajo, y generar un reporte de reposición. De estas, la más crítica en términos de frecuencia y riesgo de error es el registro de ventas, porque ocurre varias veces por hora y cualquier equivocación afecta directamente el stock real de la ferretería.

El encargado, en cambio, realiza tareas más espaciadas: revisar las órdenes de reposición generadas automáticamente por el sistema, consultar el historial de

movimientos, y eventualmente agregar productos nuevos o modificar el stock mínimo de alguno.

Contexto de uso

El sistema se usa desde una computadora de escritorio en el mostrador de la ferretería, en un entorno con bastante actividad alrededor: clientes, ruido, consultas en paralelo. Esto significa que la interfaz no puede requerir demasiada concentración para operar, y los elementos importantes como el estado del stock o los botones de acción tienen que ser visibles de forma inmediata sin necesidad de buscarlos.

Una restricción real que encontramos al diseñar es que el empleado de mostrador probablemente no tenga los dos ojos fijos en la pantalla mientras atiende. Por eso el sistema prioriza el feedback visual claro (colores, mensajes de confirmación) por encima de cualquier otro mecanismo de respuesta. Si el sistema hace algo, el usuario tiene que poder verlo aunque esté mirando para otro lado por un segundo.

- *A3 – Auditoría de usabilidad ISO 9241-11*

La norma ISO 9241-11 define usabilidad como el grado en que un sistema puede ser usado por usuarios específicos para alcanzar objetivos específicos con eficacia, eficiencia y satisfacción, en un contexto de uso determinado. Para esta auditoría evaluamos dos de esos tres criterios sobre el prototipo actual de FerreRAP.

- **Criterio 1 – Eficacia**

Definición: Grado en que el usuario logra completar la tarea correctamente, sin errores ni pasos incorrectos.

Métrica definida: Porcentaje de tareas completadas en el primer intento sin cometer errores de navegación ni de carga de datos.

Tarea evaluada: Registrar un producto nuevo en el sistema (flujo: Pantalla 1 → Pantalla 2 → Pantalla 3).

Simulación sobre el prototipo:

Al recorrer el flujo en Figma, el camino es claro: desde el inventario hay un botón visible '+ Nuevo producto' en la esquina superior derecha (Pantalla 1). Al hacer clic lleva directamente al formulario (Pantalla 2), donde todos los campos obligatorios están marcados con asterisco. Al guardar, el sistema vuelve al inventario y muestra una notificación verde de confirmación con el mensaje 'Producto creado correctamente' y el nombre del producto agregado (Pantalla 3). El contador de productos también se actualiza de 8 a 10 en tiempo real.

➢ Resultado

El flujo cumple con eficacia. El usuario puede completar la tarea sin ambigüedades porque cada pantalla tiene un único camino posible hacia adelante.

➢ Problema

El botón 'Guardar producto' aparece deshabilitado visualmente cuando el formulario está vacío, pero el prototipo no indica al usuario qué campos faltan completar. Un usuario nuevo podría no entender por qué el botón no responde.

➢ Mejora

Agregar un mensaje debajo del botón que indique los campos obligatorios pendientes cuando el usuario intenta guardar sin completar el formulario. Por ejemplo: 'Completá nombre, categoría y stock para continuar.'

- **Criterio 2 – Eficiencia**

Definición: Recursos utilizados en relación con la precisión con que se logra el objetivo. En interfaces, se mide típicamente por la cantidad de pasos o tiempo necesario para completar la tarea.

Simulación sobre el prototipo

Paso	**Acción**	**Tipo**
1	Clic en “+ Nuevo producto”	1 clic
2	Completar campo Nombre	1 campo
3	Completar campo Descripción	1 campo (opcional)

4	Seleccionar Categoría	1 campo
5	Completar Precio	1 campo
6	Completar Stock actual	1 campo
7	Completar Stock mínimo	1 campo
8	Clic en “Guardar producto”	1 clic

Total: 2 clics + 5 campos obligatorios + 1 opcional. El flujo es eficiente para la tarea que realiza.

➢ Resultado

El flujo es eficiente. No hay pantallas intermedias innecesarias ni confirmaciones redundantes. El usuario completa la tarea en una sola pantalla de formulario.

➢ Problema

El campo 'Descripción' está ubicado antes de Categoría y Precio, pero es el único campo opcional. Visualmente rompe el orden esperado (datos principales primero, datos secundarios después), lo que puede llevar al usuario a completar un campo que no es obligatorio antes de los que sí lo son.

➢ Mejora

Reordenar el formulario colocando Descripción al final, después de todos los campos obligatorios. Así el usuario sigue un flujo lógico de arriba hacia abajo completando primero lo necesario.

- Alineación con ISO 13407

ISO 13407 (actualmente reemplazada por ISO 9241-210) define un proceso de diseño centrado en el humano compuesto por 4 pasos iterativos que se repiten hasta que el sistema sea aceptable para sus usuarios. El desarrollo de FerreRAP siguió este proceso de forma implícita a lo largo del cuatrimestre:

Paso 1 — Entender y especificar el contexto de uso: Realizado en el Sprint 0, cuando se definieron los actores del sistema (empleado de mostrador y

encargado de compras), sus tareas principales y el entorno de trabajo (mostrador de ferretería, bajo presión, múltiples distracciones).

Paso 2 — Especificar los requisitos del usuario: Realizado durante el TP1, al definir las funcionalidades mínimas del sistema: registrar entradas y salidas, generar alertas automáticas, y producir reportes de reposición.

Paso 3 — Producir soluciones de diseño: El prototipo de Figma representa esta etapa. Las tres pantallas diseñadas por el UX Lead responden directamente a los requisitos identificados en los pasos anteriores.

Paso 4 — Evaluar el diseño contra los requisitos: Esta auditoría ISO 9241-11 es exactamente este paso. Al evaluar eficacia y eficiencia sobre el prototipo, identificamos dos problemas concretos y propusimos mejoras que deberían incorporarse en la próxima iteración del diseño.

El ciclo no termina acá. Las mejoras propuestas en esta auditoría alimentan una nueva iteración del prototipo, que luego volvería a evaluarse. Eso es exactamente lo que ISO 13407 plantea como proceso.

Parte B – Pruebas de Software

- *B0 – Investigación previa: técnicas de diseño*

¿Qué es una clase de equivalencia?

Una clase de equivalencia (o partición de equivalencia) es una técnica de diseño de pruebas de caja negra que divide el dominio de entrada de una función en grupos de valores que, desde el punto de vista del sistema, deben producir el mismo comportamiento. La premisa es que si una prueba con cualquier valor de la clase detecta un defecto, cualquier otro valor de esa misma clase también lo detectaría. Esto permite reducir la cantidad de casos de prueba necesarios sin perder cobertura.

Las clases se clasifican en válidas (entradas que el sistema debería aceptar) e inválidas (entradas que debería rechazar con un error controlado).

Ejemplo aplicado a registrar_salida(cantidad, motivo) de Producto:

Clase	**Rango**	**Tipo**	**Comportamiento esperado**
CE1	1 ≤ cantidad ≤ stock_actual	Válida	Stock decrementado, movimiento registrado
CE2	cantidad ≤ 0	Inválida	ValueError: 'La cantidad debe ser mayor a cero.'
CE3	cantidad > stock_actual	Inválida	ValueError: 'Stock insuficiente. Stock actual: X'

Con tres clases identificadas bastan tres casos de prueba para cubrir todos los escenarios posibles.

¿Qué es un valor límite?

El análisis de valores límite (BVA) complementa la partición de equivalencia. La experiencia empírica en pruebas de software muestra que los defectos se concentran en los bordes de las clases de equivalencia: un > codificado como >=, un índice fuera

de rango, un off-by-one. La técnica indica probar los valores exactamente en el límite de cada clase y los adyacentes inmediatos.

Ejemplo aplicado con producto de stock_actual=5

Valor	**Posición**	**Clase**	**Resultado esperado**
0	Límite inferior de CE2	Inválida	ValueError
1	Límite inferior de CE1	Válida	Stock decrementado a 4
5	Límite superior de CE1	Válida	Stock decrementado a 0
6	Límite inferior de CE3	Inválida	ValueError

Si el código tuviera un bug usando cantidad < 0 en vez de cantidad <= 0, solo el análisis de valores límite (testeando cantidad = 0) lo detectaría.

- *B1 – Casos de prueba unitaria*

Los siguientes casos de prueba cubren los métodos más críticos del dominio (models.py). El código correspondiente se encuentra en tests/unit/test_models.py del repositorio. Se implementaron seis casos cubriendo dos métodos distintos del sistema.

<u>Resumen de cobertura</u>

#	Método bajo prueba	Técnica	Tipo
TC01	`Producto.registrar_salida()`	Clase válida (CE1)	Funcional positivo
TC02	`Producto.registrar_salida()`	Valor límite inferior inválido (CE2)	Error esperado
TC03	`Producto.registrar_salida()`	Valor límite superior inválido (CE3)	Error esperado
TC04	`Producto.registrar_salida()`	Valor límite superior válido (CE1)	Observer + funcional
TC05	`Producto.registrar_entrada()`	Clase válida	Funcional positivo
TC06	`Producto.registrar_entrada()`	Observer inverso (reposición)	Integración de Observer

TC01 — Salida válida: stock se decrementa correctamente

Campo	Detalle
Método bajo prueba	`Producto.registrar_salida(cantidad, motivo)`
Técnica aplicada	Partición de equivalencia — clase válida (CE1)
Precondición	Producto con `stock_actual = 10`, `stock_minimo = 5`
Datos de entrada	`cantidad = 3`, `motivo = "Venta mostrador"`
Resultado esperado	`stock_actual` pasa a `7`; el método retorna un objeto `StockMovimiento` con `tipo = "salida"` y `cantidad = 3`; no se generan notificaciones de alerta
Criterio de pasar	`p.stock_actual == 7` y `mov.tipo == "salida"`

TC02 — Salida con cantidad cero: debe lanzar ValueError

Campo	Detalle
Método bajo prueba	`Producto.registrar_salida(cantidad, motivo)`
Técnica aplicada	Valor límite inferior inválido (límite de CE2: `cantidad = 0`)
Precondición	Producto con `stock_actual = 10`
Datos de entrada	`cantidad = 0`, `motivo = "Test"`
Resultado esperado	Se lanza `ValueError` con el mensaje `"La cantidad debe ser mayor a cero."`
Criterio de pasar	La excepción es capturada correctamente; el stock no se modifica

TC03 — Salida con cantidad mayor al stock: debe lanzar ValueError

Campo	Detalle
Método bajo prueba	`Producto.registrar_salida(cantidad, motivo)`
Técnica aplicada	Valor límite superior inválido (límite de CE3: `cantidad = stock_actual + 1`)
Precondición	Producto con `stock_actual = 5`
Datos de entrada	`cantidad = 6`, `motivo = "Test"`
Resultado esperado	Se lanza `ValueError` con el mensaje `"Stock insuficiente. Stock actual: 5"`
Criterio de pasar	La excepción es capturada; el stock permanece en `5`

TC04 — Salida que deja stock bajo mínimo activa el Observer

Campo	Detalle
Método bajo prueba	`Producto.registrar_salida()` + `Alerta.actualizar()`
Técnica aplicada	Valor límite superior válido (CE1) + verificación del patrón Observer
Precondición	Producto con `stock_actual = 5`, `stock_minimo = 5`; observador `Alerta` registrado
Datos de entrada	`cantidad = 1`, `motivo = "Venta"`
Resultado esperado	`stock_actual` pasa a `4`; como `4 < 5`, el Observer notifica; `alerta.historial` tiene exactamente una entrada con `tipo = "alerta"`
Criterio de pasar	`len(alerta.historial) == 1` y `alerta.historial[0]["tipo"] == "alerta"`

TC05 — Entrada válida: stock se incrementa correctamente

Campo	Detalle
Método bajo prueba	`Producto.registrar_entrada(cantidad, motivo)`
Técnica aplicada	Partición de equivalencia — clase válida
Precondición	Producto con `stock_actual = 3`
Datos de entrada	`cantidad = 10`, `motivo = "Reposición"`
Resultado esperado	`stock_actual` pasa a `13`; el método retorna un `StockMovimiento` con `tipo = "entrada"`
Criterio de pasar	`p.stock_actual == 13` y `mov.tipo == "entrada"`

TC06 — Entrada que normaliza el stock cierra las órdenes pendientes (Observer inverso)

Campo	Detalle
Método bajo prueba	`Producto.registrar_entrada()` + `OrdenReposicion.resolver()`
Técnica aplicada	Partición de equivalencia + verificación del patrón Observer (flujo de reposición)
Precondición	Producto con `stock_actual = 2`, `stock_minimo = 5`; observador `OrdenReposicion` con una orden pendiente preexistente
Datos de entrada	`cantidad = 10`, `motivo = "Reposición proveedor"`
Resultado esperado	`stock_actual` pasa a `12`; como `12 ≥ 5`, se llama a `resolver()` en el Observer; la orden preexistente queda con `estado = "resuelta"`
Criterio de pasar	`p.bajo_stock is False` y `orden_obs.ordenes[0]["estado"] == "resuelta"`

- *B2 – Framework de pruebas y automatización CI/CD*

<u>Framework elegido:</u> pytest

Se eligió pytest como framework de pruebas unitarias por las siguientes razones:

- **Compatibilidad con el stack:** El proyecto está desarrollado en Python 3.12, y pytest es el framework de pruebas más utilizado y mantenido del ecosistema Python.
- **Sintaxis simple:** No requiere clases ni boilerplate. Cada test es una función simple que comienza con test_, lo que facilita la lectura y el mantenimiento.
- **Aserciones nativas:** Usa el assert estándar de Python con introspección automática, mostrando el detalle del valor real vs el esperado cuando un test falla.
- **Integración directa con GitHub Actions:** Existen acciones oficiales y comunidad amplia que documentan workflows con pytest, lo que simplifica la configuración del pipeline CI/CD.
- **Sin dependencias adicionales:** Se instala con pip install pytest y no requiere configuración previa para empezar a escribir tests.

Pipeline CI/CD configurado

Se configuró un pipeline en GitHub Actions mediante el archivo .github/workflows/test.yml. El workflow se ejecuta automáticamente en cada push y pull request hacia la rama principal, mostrando los resultados en la pestaña Actions del repositorio.

El pipeline realiza los siguientes pasos:

1. Hace checkout del código del repositorio.
2. Configura Python 3.12 en el entorno.
3. Instala las dependencias necesarias (pytest y dependencias de requirements.txt).
4. Ejecuta los 6 casos de prueba unitaria con pytest tests/unit/ -v.
5. Reporta el resultado final (verde si pasan todos, rojo si alguno falla).

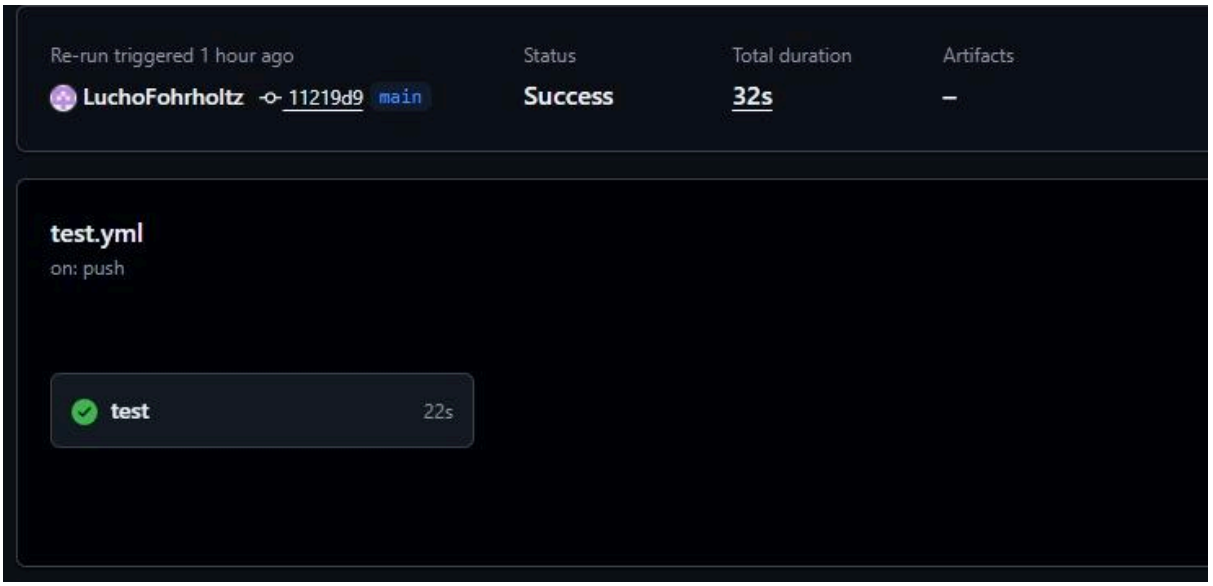
Evidencia de ejecución exitosa

Las dos capturas del workflow ejecutado correctamente (estado Success y detalle del job con todos los pasos aprobados) están publicadas en el repositorio dentro del

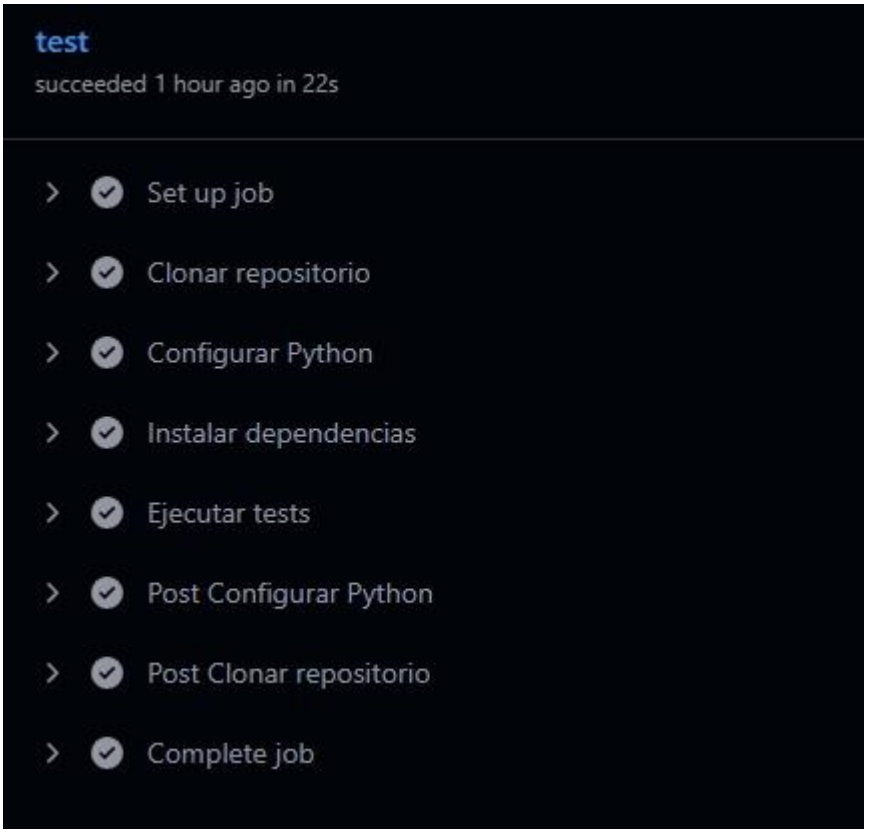
archivo docs/tp2-pruebas-unitarias.md.

https://github.com/LuchoFohrholtz/is2-2026-nipintucu/blob/main/docs/tp2-pruebas-unitarias.md

Estado Success:



Detalle del job con todos los pasos aprobados:



Video de evidencia

Se grabó un video corto mostrando los tests ejecutándose localmente y pasando en verde, junto con el workflow de GitHub Actions completándose exitosamente.

Link: *https://youtu.be/z-QnhYLdQRQ* (YouTube — no listado)

- *B3 — Diseño conceptual de pruebas de integración*

Esta sección es un diseño conceptual. Las pruebas de integración no se implementan en el TP2 sino que se planifican para etapas futuras del proyecto.

Identificación de dependencias externas

El sistema FerreRAP tiene dos dependencias externas principales que dificultan las pruebas unitarias puras y requieren estrategias de dobles de prueba:

Dependencia 1 — Flask y el cliente HTTP

Los endpoints en app.py (/api/movimientos, /api/productos, etc.) dependen del framework Flask para recibir requests HTTP, parsear JSON y devolver respuestas. En una prueba unitaria no se puede ni debe levantar un servidor real.

Estrategia de mock: Flask provee app.test_client(), un cliente de prueba que simula requests HTTP internamente sin abrir sockets de red. Se configura la aplicación en modo TESTING = True y se instancia el cliente. No se necesita ninguna biblioteca de terceros.

Dependencia 2 — Generación de archivos PDF y Excel

Los endpoints /api/reportes/<tipo>/pdf y /api/reportes/<tipo>/excel utilizan las bibliotecas reportlab y openpyxl para generar archivos binarios y devolverlos como descarga. En una prueba unitaria del endpoint, no se puede verificar fácilmente el contenido del archivo generado sin instrumentar la respuesta.

Estrategia de mock: Se utiliza unittest.mock.patch para reemplazar la función send_file de Flask por un stub que intercepta el buffer generado. La prueba verifica

que el buffer tiene contenido válido (tamaño > 0, magic bytes correctos) sin depender del filesystem ni de un browser real.

Herramienta recomendada

unittest.mock — módulo estándar de la biblioteca de Python

Justificación de la elección:

- Forma parte de la biblioteca estándar de Python desde la versión 3.3, por lo que no requiere instalación adicional ni añade dependencias al proyecto.
- Provee MagicMock, patch y patch.object que permiten reemplazar funciones, métodos o clases enteras por objetos controlables que registran cada llamada recibida.
- Con @patch('app.send_file') se puede verificar que el endpoint invoca la función con el buffer correcto y el mimetype esperado, sin que el archivo salga del proceso.
- Con MagicMock() se puede registrar un observador falso y verificar que actualizar() fue llamado exactamente una vez con el producto correcto.
- Al no necesitar servidores externos ni contenedores, los tests se ejecutan en milisegundos, lo que es compatible con el pipeline de GitHub Actions configurado en B2.

Si en el futuro se necesitara mockear servicios externos reales (bases de datos, email SMTP, APIs de terceros), se recomienda evaluar la librería responses (para HTTP) o pytest-mock (wrapper ergonómico sobre unittest.mock).

Documentación, uso de IA y estado del repositorio

Archivos generados en el repositorio:

Archivo	Contenido	Estado
docs/tp2-ui-ux.md	Parte A completa: prototipo Figma, análisis de usuario, auditoría ISO 9241-11.	Completo
docs/tp2-pruebas-unitarias .md	Parte B completa: investigación previa, 6 casos de prueba, framework, CI/CD, diseño de integración	Completo
docs/testing-strategy.md	Documento consolidado de estrategia de testing: herramientas, casos, mocks, regresión, plan de estrés.	Completo
tests/unit/test_models.py	Código real de los 6 casos de prueba unitaria implementados con pytest.	Completo
github/workflows/test.yml	Pipeline de CI/CD que ejecuta los tests automáticamente en GitHub Actions.	Completo
docs/AI_LOG.md	Registro completo del uso de IA durante el TP2 (entrada #013).	Completo

Tablero Kanban

Las tareas del TP2 se cargaron en el tablero Kanban del repositorio en GitHub Projects, organizadas por etiquetas según eje (Eje 3: HCI, Eje 4: Pruebas) y responsable. Todas las tarjetas correspondientes al TP2 fueron cumplimentadas y movidas a la columna 'Hecho' con el link al commit correspondiente como evidencia.

Uso de inteligencia artificial

El equipo utilizó herramientas de IA durante el TP2 únicamente como apoyo puntual para destrabar la redacción de algunas secciones iniciales de los documentos. Todas las decisiones técnicas (qué casos de prueba diseñar, qué herramientas elegir, qué problemas detectar en la auditoría) fueron tomadas por el equipo en base al código real del proyecto y al prototipo de Figma.

El registro completo del uso de IA, con detalle de qué herramienta se usó, qué generó, qué modificó el equipo y por qué, está documentado en el archivo docs/AI_LOG.md del repositorio (entrada #013). Cada integrante puede defender su parte del trabajo en el coloquio individual sin depender del texto generado.

Conclusión

El Trabajo Práctico N°2 permitió al equipo integrar dos ejes fundamentales de la materia: el diseño centrado en el usuario y la verificación automatizada del software. A través del prototipo en Figma y la auditoría ISO 9241-11, validamos que las decisiones de interfaz tomadas durante el TP1 son coherentes con las necesidades reales de los usuarios del sistema (empleados de mostrador y encargados de compras de una ferretería).

Por el lado de las pruebas, la implementación de seis casos unitarios sobre los métodos críticos del dominio nos permitió detectar de forma temprana cualquier regresión en la lógica de negocio. La configuración del pipeline en GitHub Actions automatiza esta verificación en cada cambio del código, lo que sienta una base sólida para escalar la cobertura en el trabajo integrador final.

Los patrones Observer y Strategy implementados en el TP1 se mantuvieron intactos durante el TP2 y demostraron ser fáciles de testear gracias al desacoplamiento que aportan. Esto valida en la práctica las decisiones de diseño tomadas en la etapa anterior y refuerza la importancia de pensar la testeabilidad desde el momento del diseño.